Table des matières

Exercices corrigés 03 — Écrire dans la base : DML et DDL ..... 1
Questions courtes ..... 1
Questions pratiques ..... 2
Diagnostics ..... 3
Questions à développer ..... 4
Mises en situation ..... 5
Corrigé ..... 7

# Exercices corrigés 03 — Écrire dans la base : DML et DDL

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font créer, remplir et modifier des tables, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui a fait des dégâts ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez quatre heures et demie pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué `[OLTP]` se traite dans PostgreSQL sur `Adventureworks`, et il écrit dans le schéma `labo03`. Un énoncé marqué `[OLAP]` se traite dans DuckDB sur l'entrepôt `adventureworksdw.duckdb`, schéma `dbo`, et il écrit dans des tables préfixées `LABO03_`. Sans marque, la question vaut pour les deux, ou se passe sur le moteur que l'énoncé nomme. Faites d'abord le nettoyage des deux labos, pour partir de bacs à sable vides, et n'écrivez jamais dans les tables d'AdventureWorks.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

---

## Questions courtes

1. Le bac à sable. Qu'est-ce qu'un bac à sable ? Quelle convention la formation suit-elle pour le délimiter sur chaque terrain, et quelle commande supprime, sur chacun, tout ce qu'un labo a créé ?

2. La copie. Que recopie `CREATE TABLE … AS`, et que ne recopie-t-il pas ? Donnez une situation où cette perte est sans importance et une où elle est grave.

3. Le type à l'insertion. Que devient 12,345 inséré dans une colonne `NUMERIC(6,2)`, et pourquoi 9 999,995 y est-il refusé ? Que se passe-t-il quand on insère 40 000 dans une colonne `SMALLINT`, puis `ABCD` dans une colonne `VARCHAR(3)`, sur PostgreSQL, puis sur DuckDB ?



© 2026 iAlexMG. Tous droits réservés.

4. Les cinq contraintes. Nommez-les, et dites pour chacune ce qu'elle interdit.
5. Les contraintes et `NULL`. Quelles contraintes laissent passer `NULL` ? Expliquez pourquoi `CHECK (prix >= 0)` accepte un livre sans prix.
6. Déclarée ou appliquée. Quelles contraintes DuckDB applique-t-il ? Et Snowflake ? Que devient, sur chacun, un doublon inséré dans une table dont la clé primaire est déclarée ?
7. La valeur par défaut. Dans quel cas une valeur `DEFAULT` s'applique-t-elle ? Que se passe-t-il si l'`INSERT` écrit explicitement `NULL` dans une colonne `NOT NULL DEFAULT 'Montréal'` ?
8. La liste de colonnes. Pourquoi faut-il nommer les colonnes d'un `INSERT` ? Décrivez deux façons dont un `INSERT` sans liste peut mal tourner.
9. `DELETE` et `TRUNCATE`. Donnez trois différences entre `DELETE FROM t` et `TRUNCATE t`.
10. La source en double. Que font `UPDATE … FROM` et `MERGE` quand la source contient deux lignes pour la même clé, sur PostgreSQL, puis sur DuckDB ? Lequel préférez-vous pour un chargement, et pourquoi ?
11. La vue et la table de travail. Qu'est-ce qu'une vue garde, et qu'est-ce qu'une table de travail garde ? Laquelle des deux peut montrer un résultat périmé, et laquelle peut montrer un résultat faux dès que la table source l'est ?

## Questions pratiques

Faites-les dans l'ordre : plusieurs questions réutilisent la table créée par une précédente. Avant chaque `UPDATE` ou `DELETE`, comptez les lignes visées, et comparez le compte rendu à ce compte.

P1. `[OLTP]` La copie de travail. Dans le schéma `labo03`, créez la table `produit` à partir de `production.product`, avec les colonnes `productid`, `name`, `productnumber`, `color`, `size`, `listprice` et `productsubcategoryid`. Vérifiez par une requête qu'elle n'a aucune contrainte, puis donnez-lui une clé primaire sur `productid`.

P2. `[OLAP]` La clé posée sur des données sales. Créez `LABO03_PRODUIT` à partir de `DimProduct`, avec `ProductKey`, `ProductAlternateKey`, `ProductSubcategoryKey`, `EnglishProductName`, `Color`, `Size` et `ListPrice`. Déclarez une clé primaire sur `ProductAlternateKey`, lisez la réponse du moteur, puis écrivez la requête qui l'explique. Concluez, et dites quelle colonne aurait dû porter la clé.

P3. `[OLTP]` Changer un type. Dans `labo03.produit`, la colonne `size` est du texte. Convertissez-la en `INTEGER`, les tailles en lettres devenant `NULL`. Avant la conversion, comptez les valeurs qui deviendront `NULL` alors qu'elles ne l'étaient pas ; après, comptez les tailles numériques et les produits sans taille.

P4. `[OLTP]` Une table faite pour durer. Créez `labo03.taux_cad`, qui range un taux de change par jour : la date, clé primaire ; le taux moyen et le taux de fin de journée, deux nombres exacts à quatre décimales, obligatoires et strictement positifs. Remplissez-la avec les taux du dollar canadien (`tocurrencycode = 'CAD'`) de `sales.currencyrate` antérieurs au 1er janvier 2024. Tentez ensuite d'insérer une seconde fois le 30 mai 2022.



© 2026 iAlexMG. Tous droits réservés.

P5. [OLTP] (avancé) Le chargement rejouable. Écrivez un MERGE qui charge dans labo03.taux_cad tous les taux du dollar canadien de sales.currencyrate : il insère les journées absentes, et ne met à jour une journée présente que si l'un de ses deux taux a changé. Vérifiez d'abord que la source n'a qu'une ligne par jour. Lancez le MERGE deux fois et comparez les deux comptes rendus.

P6. [OLTP] La correction par upsert. Le taux de fin de journée du 30 mai 2025 a été mal saisi : il vaut 1,5761, et le taux moyen de 1,5753 est juste. Écrivez la correction comme un upsert, qui insérerait la journée si elle manquait, et faites afficher par la commande elle-même la ligne écrite.

P7. [OLAP] Compter, écrire, vérifier. Le service des prix veut remplacer les prix absents de LABO03_PRODUIT par 0. Faites-le proprement : comptez les lignes visées, écrivez la modification dans une transaction, vérifiez le résultat, puis validez. Dites ensuite, chiffres à l'appui, pourquoi vous auriez déconseillé cette modification.

P8. [OLAP] UPDATE … FROM sans surprise. Ajoutez à LABO03_PRODUIT une colonne SousCategorie, et remplissez-la avec le nom anglais de la sous-catégorie, tiré de DimProductSubcategory. Prouvez d'abord que la mise à jour ne peut pas tomber dans le piège de la source en double, puis vérifiez le nombre de lignes modifiées.

P9. [OLTP] La vue mensuelle. Créez la vue labo03.taux_cad_mensuel, qui donne pour chaque mois le nombre de jours cotés et le taux moyen arrondi à quatre décimales, en nommant ses colonnes. Comptez ses lignes. Puis, dans une transaction que vous annulerez, supprimez de la table les taux de 2025, et observez ce que devient la vue.

P10. [OLTP] Faire le ménage. Tentez de supprimer la table labo03.taux_cad. Expliquez le refus, puis supprimez tout le schéma labo03 d'une seule commande, et dites combien d'objets elle emporte.

P11. [OLAP] (avancé) L'expérience de la transaction. Écrivez, sur des tables LABO03_, une séquence de commandes qui établit si un CREATE TABLE lancé au milieu d'une transaction valide ou non l'INSERT qui le précède. Dites ce que vous attendez de chaque étape, lancez la séquence, et nettoyez.

## Diagnostics

On vous montre du code qui a fait des dégâts, ou qui en fera. Dites ce qui cloche, pourquoi, et donnez le correctif.

E1. [OLTP] La promotion de trop. Une analyste veut baisser de 10 % le prix des trois casques Sport-100 dans sa copie de travail. Elle lance, dans pgAdmin :

```
UPDATE labo03.produit SET listprice = listprice * 0.9;
```

Le compte rendu affiche UPDATE 504. Qu'est-il arrivé ? Que peut-elle faire maintenant, et quelles habitudes l'auraient protégée ?

E2. Le MERGE qui plante. Depuis un an, sur l'entrepôt Snowflake d'une entreprise, un traitement de nuit met à jour les courriels de la dimension client :

```
UPDATE DIM_CLIENT AS d
SET Courriel = s.Courriel
FROM STG_CLIENT AS s
WHERE s.CustomerKey = d.CustomerKey;
```



© 2026 iAlexMG. Tous droits réservés.

Un collègue le récrit en `MERGE` pour insérer aussi les nouveaux clients. Dès le premier soir, le traitement échoue sur une erreur de ligne en double dans la source. L'équipe propose de revenir à l'ancien `UPDATE`, « qui, lui, n'a jamais planté ». Diagnostiquez, dites ce que vous faites, et ce qui se serait passé si le même `MERGE` avait tourné sur DuckDB.

E3. Le chiffre d'affaires qui double. Sur un entrepôt Snowflake, après la relance d'un chargement tombé en panne, un rapport montre des ventes doublées pour une partie des clients. La dimension `DIM_CLIENT` a pourtant une clé primaire déclarée sur `CustomerKey`, et le responsable affirme que « la base aurait refusé les doublons ». Expliquez le mécanisme, et écrivez la requête qui le prouve.

E4. La sauvegarde qui a tout validé. Sur un entrepôt Snowflake, avant une purge, un analyste écrit :

```
BEGIN;
DELETE FROM CLIENT WHERE DateFirstPurchase < '2014-01-15';
CREATE OR REPLACE TABLE CLIENT_SAUVEGARDE AS SELECT * FROM CLIENT;
SELECT COUNT(*) FROM CLIENT;
ROLLBACK;
```

Il comptait regarder le compte, puis tout annuler. Après le `ROLLBACK`, les clients supprimés ne sont pas revenus, et la sauvegarde ne les contient pas non plus. Expliquez les deux erreurs, récrivez la séquence, et dites laquelle des deux erreurs aurait subsisté sur DuckDB.

E5. `[OLTP]` Les conversions qui dérivent. Une table de taux de change a été créée ainsi, puis chargée avec les taux du dollar canadien de `sales.currencyrate` :

```
CREATE TABLE labo03.taux (
    jour DATE          PRIMARY KEY,
    taux NUMERIC(6,2)  NOT NULL
);
```

Elle sert à convertir des factures en dollars canadiens, et les montants convertis s'écartent de ceux du service financier, de plusieurs centaines de dollars sur les grosses factures. Aucune erreur n'a jamais été levée. Diagnostiquez, puis corrigez sans perdre davantage d'information.

E6. `[OLTP]` L'historique inversé. Un script mensuel archive les promotions :

```
INSERT INTO labo03.promotion_historique
SELECT specialofferid, description, discountpct, type, category, startdate, enddate
FROM sales.specialoffer;
```

Il fonctionne depuis deux ans. Le mois dernier, un collègue a recréé la table d'archive « à l'identique », en plaçant la catégorie avant le type, dans un ordre qu'il trouvait plus logique. Depuis, le rapport par catégorie affiche des catégories nommées « Volume Discount » et « Seasonal Discount ». Expliquez, et corrigez le script.

## Questions à développer

Un court paragraphe de raisonnement, pas un mot juste.



© 2026 iAlexMG. Tous droits réservés.

A. La contrainte qu'on n'applique pas. D'après sa documentation, Snowflake déclare des clés primaires, des contraintes `UNIQUE` et des clés étrangères sans les appliquer, alors que DuckDB, lui aussi un moteur d'entrepôt, les applique. Défendez le choix de Snowflake du point de vue d'un entrepôt, puis dites ce qu'il exige d'une équipe qui charge des données, et où ce contrôle doit vivre.

B. `DELETE` contre `TRUNCATE`. Comparez les deux commandes du point de vue du volume, des clés étrangères, des transactions et de ce qu'on veut garder. Donnez une situation où `TRUNCATE` est le bon choix et une où ce serait une faute.

C. Le déploiement d'une migration. Une migration en trois étapes ajoute une colonne, la remplit par un `UPDATE`, puis la déclare `NOT NULL`. La troisième étape échoue, parce que quelques lignes sont restées vides. Décrivez l'état de la base après l'échec sur PostgreSQL ou DuckDB, puis sur Snowflake, et ce que chaque moteur vous oblige à prévoir avant de lancer la migration.

## Mises en situation

On décrit une entreprise et une question métier. À vous de choisir l'approche, de l'écrire, et surtout de la justifier. Il y a plusieurs bonnes réponses, mais il y a des réponses indéfendables.

M1. Coopérative laitière du Bas-Saint-Laurent, le chargement des livraisons. Chaque nuit, la coopérative reçoit un fichier des livraisons de lait des sept derniers jours, parce que les fermes corrigent parfois une livraison déjà transmise. La table des livraisons de l'entrepôt compte cinq ans d'historique. Deux développeurs s'opposent : l'un veut supprimer les sept derniers jours et les réinsérer, l'autre veut un `MERGE` sur le couple ferme et jour.

Tranchez. Traitez trois cas : une livraison que la ferme a retirée du fichier, un fichier livré deux fois la même nuit, et un fichier qui contient deux lignes pour la même ferme et le même jour.

M2. Agence de voyages de Québec, l'unicité que plus personne ne vérifie. L'agence a migré sa dimension client d'une base transactionnelle vers Snowflake. Sur l'ancienne, la clé primaire bloquait les doublons ; sur Snowflake, trois mois plus tard, 2 % des clients sont en double faux.

Dites où et comment vous contrôleriez désormais l'unicité, à quel moment du chargement, et ce que vous feriez d'un chargement qui échoue au contrôle. La formation Analytics Engineering fournit l'outil ; ici, on veut la stratégie.

M3. Centre de services scolaire des Hautes-Laurentides, le tableau de bord des absences. Le tableau de bord lit une requête qui joint les absences, les élèves et les horaires, et qui prend quatre minutes. Une conseillère propose d'en faire une vue, un analyste une table de travail recalculée chaque nuit. Les directions consultent le tableau chaque matin, et les absences se saisissent toute la journée.

Tranchez, en disant ce que chaque option montre à 10 h, ce qu'elle coûte, et ce qui se passe le jour où la requête contient une erreur.

M4. `(avancé)` Transporteur de Saint-Hyacinthe, la colonne à corriger en production. Dans la base PostgreSQL de répartition, le poids des envois est rangé en `VARCHAR`, avec des valeurs comme « 1250 », « 980 kg » et « inconnu ». Deux vues et un rapport de facturation lisent la colonne. On vous demande de la passer en `NUMERIC(8,2)` sans interrompre la facturation.

Décrivez vos étapes dans l'ordre, les commandes que vous emploieriez, ce que vous vérifiez entre deux étapes, et ce qui vous permet de revenir en arrière.



© 2026 iAlexMG. Tous droits réservés.

M5. Quincaillerie en ligne de Sherbrooke, 14 h 05. Un employé vient de lancer `UPDATE produit SET prix = 9.99;` sur la base PostgreSQL du site, en oubliant le `WHERE`. Le site affiche tous les articles à 9,99 $, et des commandes commencent à entrer.

Dites ce que vous faites dans les cinq premières minutes, puis dans l’heure. Dites ce qui changerait si la même erreur avait frappé la table des prix de l’entrepôt Snowflake, qui alimente les rapports. Et dites ce que vous changez ensuite pour que ça ne se reproduise pas.



© 2026 iAlexMG. Tous droits réservés.

## Corrigé

La numérotation est celle des énoncés.

---

### Questions courtes

1. Un bac à sable est un espace de travail à soi, où une erreur ne casse que son propre travail : on n'écrit pas là où d'autres lisent. Côté PostgreSQL, la formation crée un schéma par cours, `labo03`, et `DROP SCHEMA labo03 CASCADE` supprime tout son contenu d'une commande. Côté DuckDB, elle préfixe les objets, `LABO03_`, dans le schéma `dbo`, et le nettoyage énumère un `DROP TABLE IF EXISTS`, un `DROP VIEW IF EXISTS` ou un `DROP SEQUENCE IF EXISTS` par objet. Le préfixe oblige à tenir la liste : c'est le prix de garder les tables de travail à côté des tables qu'elles lisent.

*Théorie : `Theorie_03_SQL.md`, sections 1 et 9.*

2. `CREATE TABLE … AS` recopie les noms et les types des colonnes, et les données. Il ne recopie ni la clé primaire, ni les `NOT NULL`, ni les valeurs par défaut, ni l'identité, ni les `CHECK`, ni les clés étrangères. La perte est sans importance pour une table de travail jetable, comme la copie des 16 promotions du labo, qu'on lit une heure puis qu'on supprime. Elle est grave quand la copie devient une table de référence dans laquelle un traitement écrit : plus rien n'empêche un doublon, une absence ou un numéro saisi à la main, et personne ne s'en aperçoit, puisque rien ne lève d'erreur.

*Théorie : `Theorie_03_SQL.md`, section 2.*

3. 12,345 devient 12,35, sans message : le type arrondit à deux décimales. 9 999,995 s'arrondit à 10 000,00, qui a cinq chiffres avant la virgule, alors que `NUMERIC(6,2)` n'en permet que quatre : « champ numérique en dehors des limites » ; DuckDB le refuse aussi. 40 000 dans un `SMALLINT` est refusé sur les deux terrains, parce que ce type s'arrête à 32 767 : « smallint en dehors des limites » sur PostgreSQL, `out of range for the destination type INT16` sur DuckDB. `ABCD` dans un `VARCHAR(3)` les sépare : PostgreSQL le refuse, « valeur trop longue pour le type character varying(3) », et DuckDB le range en entier, quatre caractères, parce qu'il ne vérifie jamais la longueur déclarée. Snowflake, d'après sa documentation, ferait l'inverse de DuckDB sur les deux : longueur vérifiée, et 40 000 accepté, son `SMALLINT` n'étant qu'un autre nom de `NUMBER(38,0)`.

*Théorie : `Theorie_03_SQL.md`, section 2.*

4. `NOT NULL` interdit l'absence de valeur. `PRIMARY KEY` interdit deux lignes avec la même clé et une clé absente. `UNIQUE` interdit deux lignes avec la même valeur, mais permet l'absence. La clé étrangère, `REFERENCES`, interdit une valeur qui n'existe pas dans la table visée, et, dans l'autre sens, la suppression d'une ligne encore référencée. `CHECK` interdit une ligne qui rend sa condition fausse.

*Théorie : `Theorie_03_SQL.md`, section 3.*

5. `UNIQUE` et `CHECK` laissent passer `NULL`, et une clé étrangère aussi quand sa colonne l'accepte. `CHECK` ne rejette que ce qui est faux. Pour un livre sans prix, `prix >= 0` compare `NULL` à 0, et le résultat n'est ni vrai ni faux, il est inconnu : la ligne passe. Si le prix est obligatoire, c'est un `NOT NULL` qui doit le dire, en plus du `CHECK`.

*Théorie : `Theorie_03_SQL.md`, section 3.*



© 2026 iAlexMG. Tous droits réservés.

6. DuckDB applique les cinq, comme PostgreSQL : Mindy Luo inscrite une seconde fois dans le labo OLAP est refusée, `Duplicate key "CustomerKey: 11349" violates primary key constraint`. Snowflake, d'après sa documentation, n'applique que `NOT NULL` et `CHECK` sur une table ordinaire : il enregistre la clé primaire, `UNIQUE` et la clé étrangère sans les appliquer, sauf sur ses tables hybrides. Un doublon inséré dans une table à clé primaire déclarée y entre sans erreur, et seule une requête de contrôle, `GROUP BY` la clé avec `HAVING COUNT(*) > 1`, le révèle.

*Théorie : `Theorie_03_SQL.md`, section 3.*

7. Une valeur `DEFAULT` s'applique quand l'`INSERT` ne nomme pas la colonne, ou quand il écrit le mot `DEFAULT` à la place d'une valeur. Un `NULL` écrit explicitement est une valeur fournie : il ne déclenche pas la valeur par défaut. Dans une colonne `NOT NULL DEFAULT 'Montréal'`, la ligne est refusée, « une valeur NULL viole la contrainte NOT NULL de la colonne « ville » ». Sans le `NOT NULL`, ce serait pire : la ligne entrerait avec une ville vide, en silence.

*Théorie : `Theorie_03_SQL.md`, section 4.*

8. Parce que, sans liste, le moteur associe les valeurs aux colonnes par position, jamais par nom. La liste rend la commande insensible à l'ordre des colonnes, à leur nombre et aux changements de structure. Première façon de mal tourner : deux colonnes du même type interverties, et les valeurs entrent dans la mauvaise colonne sans erreur, comme le `type` et la `category` du labo, ou l'emprunt 2 attribué au membre 4 dans la théorie. Seconde façon : la structure change. Sur PostgreSQL, un `INSERT` écrit avant l'ajout d'une colonne continue de passer et laisse la nouvelle colonne vide ; sur DuckDB, il échoue ; et si la table est recréée dans un autre ordre, on retombe sur l'inversion silencieuse.

*Théorie : `Theorie_03_SQL.md`, section 4.*

9. Trois différences parmi d'autres. `DELETE` choisit ses lignes par un `WHERE`, `TRUNCATE` vide toute la table. `DELETE` travaille ligne par ligne et reste lent sur une grosse table, `TRUNCATE` vide la table d'un bloc. Face à une clé étrangère, `DELETE` ne refuse que les lignes encore référencées, alors que `TRUNCATE` refuse la commande entière. On peut ajouter que `TRUNCATE … RESTART IDENTITY` remet le compteur d'identité à zéro sur PostgreSQL, et que le Cours 01 range `TRUNCATE` dans le DDL.

*Théorie : `Theorie_03_SQL.md`, section 5.*

10. `UPDATE … FROM` accepte la source en double sur les deux terrains, sans erreur, et applique l'une des valeurs, sans qu'on sache laquelle : la première dans l'exemple de la théorie, la seconde dans le labo OLTP, l'une ou l'autre selon le lancement dans le labo OLAP. `MERGE` se sépare : PostgreSQL le refuse, « MERGE command cannot affect row a second time », et n'écrit rien ; DuckDB l'accepte et garde, lui aussi, l'une des valeurs. Pour un chargement, je préfère quand même `MERGE` : il insère les lignes nouvelles dans la même commande, il se rejoue sans créer de doublon, et, sur PostgreSQL, il échoue bruyamment sur un défaut que l'`UPDATE … FROM` cache. Mais comme ce garde-fou dépend du moteur, on vérifie l'unicité de la clé dans la source avant d'écrire, partout.

*Théorie : `Theorie_03_SQL.md`, sections 5 et 6.*

11. Une vue garde une requête, pas de données : chaque interrogation relit les tables. Une table de travail garde une photo des données, prise au moment où on l'a calculée. La table de travail peut montrer un résultat périmé, comme la colonne `Segment` du labo OLAP, qui marquait encore 67 clients aisés alors que la vue n'en comptait plus que 66. La vue, elle, n'est jamais périmée, mais elle montre un résultat faux à la seconde où la source l'est : un `UPDATE` sans `WHERE` a vidé la vue des emprunts en retard sans qu'on y touche.



© 2026 iAlexMG. Tous droits réservés.

*Théorie : `Theorie_03_SQL.md`, section 7.*

## Questions pratiques

P1. [OLTP]

```sql
CREATE SCHEMA IF NOT EXISTS labo03;

CREATE TABLE labo03.produit AS
SELECT productid, name, productnumber, color, size, listprice, productsubcategoryid
FROM production.product;

SELECT conname, contype
FROM pg_constraint
WHERE conrelid = 'labo03.produit'::regclass;

ALTER TABLE labo03.produit ADD PRIMARY KEY (productid);
```

`SELECT 504`, puis aucune ligne dans `pg_constraint` : la copie n'a ni clé ni `NOT NULL`. L'`ALTER` passe, parce que les 504 `productid` sont présents et uniques : PostgreSQL vérifie les lignes en place avant d'accepter la clé. Relancée ensuite, la requête sur le catalogue montre `produit_pkey`, et, sur PostgreSQL 18, la contrainte `NOT NULL` que la clé primaire a ajoutée à la colonne.

*Théorie : `Theorie_03_SQL.md`, sections 1, 2 et 3.*

P2. [OLAP]

```sql
CREATE OR REPLACE TABLE LABO03_PRODUIT AS
SELECT ProductKey, ProductAlternateKey, ProductSubcategoryKey, EnglishProductName,
       Color, Size, ListPrice
FROM DimProduct;

ALTER TABLE LABO03_PRODUIT ADD PRIMARY KEY (ProductAlternateKey);

SELECT COUNT(*) AS lignes, COUNT(DISTINCT ProductAlternateKey) AS codes
FROM LABO03_PRODUIT;

SELECT ProductAlternateKey, COUNT(*) AS versions
FROM LABO03_PRODUIT
GROUP BY ProductAlternateKey
HAVING COUNT(*) > 1
ORDER BY versions DESC;
```



© 2026 iAlexMG. Tous droits réservés.

L'`ALTER` est refusé : `Data contains duplicates on indexed column(s)`. DuckDB vérifie les lignes en place avant d'accepter la clé, comme PostgreSQL l'a fait en P1. Les deux requêtes suivantes disent pourquoi : 606 lignes pour 504 codes, et 77 codes présents plusieurs fois, 52 deux fois et 25 trois fois, soit 102 lignes de trop pour une clé primaire. La cause est connue depuis le Cours 02 : `DimProduct` garde plusieurs versions d'un même produit. `ProductAlternateKey` est le code du produit, unique par version seulement ; la clé de la table est `ProductKey`, unique sur les 606 lignes. Sur un entrepôt qui n'applique pas les clés, comme Snowflake, l'`ALTER` aurait passé sans rien vérifier, et seule la requête de contrôle aurait montré le défaut.

*Théorie : `Theorie_03_SQL.md`, sections 2 et 3.*

P3. `[OLTP]`

```sql
SELECT COUNT(*) AS deviendront_null
FROM labo03.produit
WHERE size IS NOT NULL
  AND size !~ '^[0-9]+$';

ALTER TABLE labo03.produit
ALTER COLUMN size TYPE INTEGER
USING CASE WHEN size ~ '^[0-9]+$' THEN CAST(size AS INTEGER) END;

SELECT COUNT(size) AS tailles_numeriques, COUNT(*) - COUNT(size) AS sans_taille
FROM labo03.produit;
```

34 tailles en lettres deviendront `NULL`. Après la conversion, 177 tailles numériques et 327 produits sans taille : les 293 qui n'en avaient pas, plus les 34. `!~` veut dire « ne correspond pas à l'expression régulière ». Écrite avec un simple `CAST`, la conversion échoue sur « M » et ne change rien. Le compte préalable est la leçon du `TRY_CAST` au Cours 02 : on sait ce qu'on perd avant de le perdre. Sur une vraie table, je garderais d'ailleurs la colonne d'origine et j'ajouterais la colonne convertie à côté, plutôt que de détruire les lettres.

*Théorie : `Theorie_03_SQL.md`, sections 2 et 7.*

P4. `[OLTP]`

```sql
CREATE TABLE labo03.taux_cad (
    jour          DATE          PRIMARY KEY,
    taux_moyen    NUMERIC(10,4) NOT NULL CHECK (taux_moyen > 0),
    taux_fin_jour NUMERIC(10,4) NOT NULL CHECK (taux_fin_jour > 0)
);

INSERT INTO labo03.taux_cad (jour, taux_moyen, taux_fin_jour)
SELECT CAST(currencyratedate AS DATE), averagerate, endofdayrate
FROM sales.currencyrate
WHERE tocurrencycode = 'CAD'
  AND currencyratedate < '2024-01-01';
```



© 2026 iAlexMG. Tous droits réservés.

```sql
INSERT INTO labo03.taux_cad (jour, taux_moyen, taux_fin_jour)
VALUES ('2022-05-30', 1.4641, 1.4683);
```

`INSERT 0 581`, du 30 mai 2022 au 31 décembre 2023. La seconde insertion est refusée : « la valeur d'une clé dupliquée rompt la contrainte unique « taux_cad_pkey » ». `NUMERIC(10,4)` n'est pas un hasard : les taux de la source ont quatre décimales, de 1,4357 à 1,6154, et un type à deux décimales les arrondirait sans prévenir (voir E5). La conversion en `DATE` ne perd rien, puisque tous les taux sont datés à minuit.

*Théorie : `Theorie_03_SQL.md`, sections 2, 3 et 4.*

P5. `[OLTP]` `(avancé)`

```sql
SELECT CAST(currencyratedate AS DATE) AS jour, COUNT(*) AS lignes
FROM sales.currencyrate
WHERE tocurrencycode = 'CAD'
GROUP BY CAST(currencyratedate AS DATE)
HAVING COUNT(*) > 1;

MERGE INTO labo03.taux_cad AS t
USING (SELECT CAST(currencyratedate AS DATE) AS jour, averagerate, endofdayrate
       FROM sales.currencyrate
       WHERE tocurrencycode = 'CAD') AS s
   ON s.jour = t.jour
WHEN MATCHED AND (t.taux_moyen <> s.averagerate OR t.taux_fin_jour <> s.endofdayrate) THEN
    UPDATE SET taux_moyen = s.averagerate, taux_fin_jour = s.endofdayrate
WHEN NOT MATCHED THEN
    INSERT (jour, taux_moyen, taux_fin_jour)
    VALUES (s.jour, s.averagerate, s.endofdayrate);
```

Le contrôle ne rend aucune ligne : un taux par jour. Le premier `MERGE` répond `MERGE 516`, les 516 journées de 2024 et 2025 insérées, aucune mise à jour ; le second répond `MERGE 0`. La table compte 1 097 journées, jusqu'au 30 mai 2025. La condition ajoutée à `WHEN MATCHED` fait la différence : sans elle, le second passage réécrirait les 1 097 lignes à l'identique et répondrait `MERGE 1097`, ce qui rend le compte rendu inutile pour savoir si quelque chose a changé. Si les colonnes acceptaient `NULL`, `<>` raterait le passage d'une absence à une valeur, et on écrirait `IS DISTINCT FROM`.

*Théorie : `Theorie_03_SQL.md`, section 6.*

P6. `[OLTP]`

```sql
INSERT INTO labo03.taux_cad (jour, taux_moyen, taux_fin_jour)
VALUES ('2025-05-30', 1.5753, 1.5761)
ON CONFLICT (jour) DO UPDATE
    SET taux_fin_jour = EXCLUDED.taux_fin_jour
RETURNING jour, taux_moyen, taux_fin_jour;
```



© 2026 iAlexMG. Tous droits réservés.

Une ligne revient : le 30 mai 2025, 1,5753 et 1,5761, au lieu du 1,5758 chargé. Le `VALUES` fournit les deux taux, pour le cas où la journée manquerait ; la branche `DO UPDATE` ne corrige que le taux connu comme faux. PostgreSQL 18 permet en plus de voir l'avant et l'après dans la même commande : `RETURNING old.taux_fin_jour AS avant, new.taux_fin_jour AS apres`.

*Théorie : `Theorie_03_SQL.md`, section 6.*

P7. [OLAP]

```sql
SELECT COUNT(*) AS sans_prix FROM LABO03_PRODUIT WHERE ListPrice IS NULL;

BEGIN;
UPDATE LABO03_PRODUIT SET ListPrice = 0 WHERE ListPrice IS NULL;
SELECT COUNT(*) AS sans_prix FROM LABO03_PRODUIT WHERE ListPrice IS NULL;
COMMIT;
```

211 lignes comptées, 211 modifiées, 0 après. Je l'aurais déconseillé, parce qu'un 0 n'est pas une absence, c'est une sentinelle (Cours 01, section 9). Le prix moyen de la table passe de 747,66 $, calculé sur les 395 lignes qui ont un prix, à 487,34 $, calculé sur 606 lignes dont 211 à zéro, et le prix minimum devient 0. Toute analyse qui oublie de filtrer la sentinelle se trompe. Pour afficher 0 à la place d'un prix absent, on écrit `COALESCE(ListPrice, 0)` au moment de lire, et la donnée reste intacte.

*Théorie : `Theorie_03_SQL.md`, sections 5 et 8.*

P8. [OLAP]

```sql
SELECT ProductSubcategoryKey, COUNT(*) AS lignes
FROM DimProductSubcategory
GROUP BY ProductSubcategoryKey
HAVING COUNT(*) > 1;

ALTER TABLE LABO03_PRODUIT ADD COLUMN SousCategorie VARCHAR;

UPDATE LABO03_PRODUIT AS p
SET SousCategorie = s.EnglishProductSubcategoryName
FROM DimProductSubcategory AS s
WHERE s.ProductSubcategoryKey = p.ProductSubcategoryKey;
```

Le contrôle ne rend rien : 37 sous-catégories, 37 clés différentes, donc chaque produit trouve au plus une ligne dans la source. L'`UPDATE` modifie 397 lignes, les 606 moins les 209 produits sans sous-catégorie (Cours 01, section 7). Ces 209 gardent une `SousCategorie` à `NULL` : un `UPDATE … FROM` ne touche que les lignes qui trouvent leur correspondance, comme une jointure interne.

*Théorie : `Theorie_03_SQL.md`, sections 5 et 7.*

P9. [OLTP]



© 2026 iAlexMG. Tous droits réservés.

```sql
CREATE VIEW labo03.taux_cad_mensuel AS
SELECT CAST(DATE_TRUNC('month', jour) AS DATE) AS mois,
       COUNT(*)                                AS jours,
       ROUND(AVG(taux_moyen), 4)               AS taux_moyen
FROM labo03.taux_cad
GROUP BY CAST(DATE_TRUNC('month', jour) AS DATE);

SELECT COUNT(*) AS mois FROM labo03.taux_cad_mensuel;

BEGIN;
DELETE FROM labo03.taux_cad WHERE jour >= '2025-01-01';
SELECT COUNT(*) AS mois FROM labo03.taux_cad_mensuel;
ROLLBACK;
```

37 mois, de mai 2022 (2 jours cotés, 1,4651) à mai 2025 (30 jours, 1,5660). Le `DELETE` retire 150 journées, et la vue tombe à 32 mois, sans qu'on l'ait touchée ; après le `ROLLBACK`, elle revient à 37. La colonne `jours` n'est pas décorative : elle montre que la moyenne de mai 2022 repose sur deux journées et ne se compare pas aux autres. La conversion en `DATE` évite l'horodatage avec fuseau que `DATE_TRUNC` rend sur une date (Cours 02, section 6).

*Théorie : `Theorie_03_SQL.md`, sections 7 et 8.*

P10. `[OLTP]`

```sql
DROP TABLE labo03.taux_cad;

DROP SCHEMA labo03 CASCADE;
```

Le premier est refusé : « n'a pas pu supprimer table labo03.taux_cad car d'autres objets en dépendent », la vue `taux_cad_mensuel`. Le second annonce « DROP cascade sur 3 autres objets » : les tables `produit` et `taux_cad`, et la vue. Le `CASCADE` est justifié ici, parce que tout le bac à sable est jetable et que le refus précédent a donné la liste de ce qu'on emporte. Si le nettoyage d'un labo a été sauté, le compte est plus élevé.

*Théorie : `Theorie_03_SQL.md`, sections 1 et 7.*

P11. `[OLAP]` (avancé)

```sql
CREATE OR REPLACE TABLE LABO03_TX (x INTEGER);
INSERT INTO LABO03_TX VALUES (1);            -- témoin, validé hors transaction

BEGIN;
INSERT INTO LABO03_TX VALUES (2);            -- l'écriture qu'on veut pouvoir annuler
CREATE OR REPLACE TABLE LABO03_TX_DDL (x INTEGER);
ROLLBACK;

SELECT x FROM LABO03_TX ORDER BY x;          -- 1 et 2 si le DDL a validé, 1 sinon
```



© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT COUNT(*) AS labo03_tx_ddl            -- 1 si la table existe, 0 sinon
FROM information_schema.tables
WHERE table_name = 'LABO03_TX_DDL';

DROP TABLE IF EXISTS LABO03_TX;
DROP TABLE IF EXISTS LABO03_TX_DDL;
```

La séquence isole une seule question. Le témoin, validé avant la transaction, prouve que la table se lit ; l'`INSERT` de la valeur 2 est la seule écriture dont le sort dépend du DDL. Si la requête finale ne rend que 1, le `CREATE TABLE` n'a rien validé ; si elle rend 1 et 2, il a validé la transaction en cours. Sur DuckDB, elle rend 1 seul, et le catalogue ne connaît aucune `LABO03_TX_DDL` : le `ROLLBACK` a tout annulé, la création de table comprise, comme PostgreSQL l'aurait fait. Sur Snowflake, d'après sa documentation, on obtiendrait 1 et 2, et une table bien présente.

*Théorie : `Theorie_03_SQL.md`, section 8.*

### Diagnostics

E1. `[OLTP]` Le `WHERE` manque : les 504 produits ont perdu 10 %, et non les 3 casques. pgAdmin était en autovalidation, la commande est validée, et aucun `ROLLBACK` ne la rattrape. Comme il s'agit d'une copie, la réparation est simple : on relit les prix d'origine, puis on refait la bonne commande.

```sql
UPDATE labo03.produit AS c
SET listprice = p.listprice
FROM production.product AS p
WHERE p.productid = c.productid;

SELECT COUNT(*) FROM labo03.produit WHERE name LIKE 'Sport-100%';    -- 3

BEGIN;
UPDATE labo03.produit
SET listprice = ROUND(listprice * 0.9, 2)
WHERE name LIKE 'Sport-100%';                                        -- UPDATE 3
COMMIT;
```

Sur une vraie table sans copie d'origine, il ne resterait que la sauvegarde. Trois habitudes l'auraient protégée : compter d'abord, ouvrir une transaction, et comparer le compte rendu au compte avant de valider.

*Théorie : `Theorie_03_SQL.md`, sections 5 et 8.*

E2. La source `STG_CLIENT` contient des clients en double, deux changements de courriel le même jour ou un fichier chargé deux fois. L'`UPDATE … FROM` les avalait depuis un an en retenant l'une des valeurs, parfois l'ancienne adresse, sans rien dire. Le `MERGE` n'a pas créé le problème, il l'a révélé. Revenir à l'`UPDATE`, ce serait remettre un bandeau sur les yeux. Je mesure d'abord :

```sql
SELECT CustomerKey, COUNT(*) AS lignes
FROM STG_CLIENT
```



© 2026 iAlexMG. Tous droits réservés.

```sql
GROUP BY CustomerKey
HAVING COUNT(*) > 1;
```

Puis on fixe une règle écrite, par exemple « la modification la plus récente gagne », appliquée dans la requête source du `MERGE` (le Cours 04 donne l'outil, `ROW_NUMBER` et `QUALIFY`). Enfin, on cherche pourquoi la zone de préparation reçoit des doublons, et on garde le contrôle d'unicité avant chaque chargement. Sur DuckDB, le même `MERGE` n'aurait pas planté : il accepte la source en double et garde l'une des valeurs, exactement comme l'ancien `UPDATE`. Le défaut serait resté caché, et seul le contrôle d'unicité l'aurait montré.

*Théorie : `Theorie_03_SQL.md`, sections 5 et 6.*

E3. La relance a inséré une seconde fois les lignes de la dimension : un `INSERT … SELECT` n'est pas rejouable. La clé primaire déclarée n'a rien refusé, parce que Snowflake ne l'applique pas ; sur DuckDB ou PostgreSQL, la relance aurait échoué sur la clé, et rien n'aurait été doublé. Chaque vente d'un client en double trouve maintenant deux lignes dans la dimension, et la jointure la compte deux fois, le piège de la jointure qui multiplie les lignes (Cours 01). La preuve :

```sql
SELECT CustomerKey, COUNT(*) AS lignes
FROM DIM_CLIENT
GROUP BY CustomerKey
HAVING COUNT(*) > 1;
```

Et son effet : le nombre de lignes de la table de faits comparé au nombre de lignes de sa jointure avec `DIM_CLIENT`, qui devraient être égaux. La réparation reconstruit la dimension sans doublons ; un `SELECT DISTINCT *` suffit si les lignes doublées sont identiques, pas si une colonne comme la date de chargement les distingue. La prévention : un chargement rejouable, par `MERGE`, et un test d'unicité après chaque chargement.

*Théorie : `Theorie_03_SQL.md`, sections 3 et 6.*

E4. Deux erreurs. La première : `CREATE OR REPLACE TABLE` est un DDL, et sur Snowflake il valide d'office la transaction en cours avant de s'exécuter. Le `DELETE` a donc été validé à ce moment-là, et le `ROLLBACK` n'avait plus rien à annuler. La seconde : la sauvegarde a été prise après la suppression, elle contient la table déjà purgée. La séquence correcte sauvegarde d'abord, hors transaction, puis écrit sans aucun DDL dans la transaction :

```sql
CREATE OR REPLACE TABLE CLIENT_SAUVEGARDE AS
SELECT * FROM CLIENT;

SELECT COUNT(*) FROM CLIENT WHERE DateFirstPurchase < '2014-01-15';

BEGIN;
DELETE FROM CLIENT WHERE DateFirstPurchase < '2014-01-15';
SELECT COUNT(*) FROM CLIENT;
ROLLBACK;
```

Le `ROLLBACK` final devient un `COMMIT` une fois le compte vérifié. Pour récupérer les lignes perdues, le voyage dans le temps du Cours 11 relit la table telle qu'elle était avant la purge. Sur DuckDB ou PostgreSQL, la première erreur disparaît, la séquence d'origine étant annulée en entier, DDL compris ; la seconde subsiste, et la sauvegarde aurait été tout aussi inutile.



© 2026 iAlexMG. Tous droits réservés.

*Théorie : `Theorie_03_SQL.md`, sections 2 et 8.*

E5. `[OLTP]` `NUMERIC(6,2)` arrondit chaque taux à deux décimales, sans erreur : le taux moyen du 30 mai 2025, 1,5753, est rangé 1,58. Sur les 1 097 journées, 1 077 ont été altérées, avec un écart allant jusqu'à 0,0050 ; les 20 autres avaient déjà deux zéros en fin de taux. Sur une facture de 100 000 $ US, la conversion donne 158 000 $ au lieu de 157 530 $, 470 $ d'écart. Changer le type ne suffit pas : après `ALTER COLUMN taux TYPE NUMERIC(10,4)`, la valeur vaut 1,5800, les décimales perdues ne reviennent pas. Il faut élargir le type, puis recharger depuis la source, dont on sait (P5) qu'elle n'a qu'un taux par jour :

```sql
ALTER TABLE labo03.taux ALTER COLUMN taux TYPE NUMERIC(10,4);

UPDATE labo03.taux AS t
SET taux = c.averagerate
FROM sales.currencyrate AS c
WHERE c.tocurrencycode = 'CAD'
  AND CAST(c.currencyratedate AS DATE) = t.jour;
```

`UPDATE 1097`, et le 30 mai 2025 revient à 1,5753. Reste à recalculer les conversions déjà faites avec les taux arrondis.

*Théorie : `Theorie_03_SQL.md`, sections 2 et 5.*

E6. `[OLTP]` L'`INSERT` n'a pas de liste de colonnes : il range les valeurs par position. La table recréée place `category` avant `type`, donc le `type` du `SELECT` tombe dans `category`, et inversement. Les deux colonnes sont du texte, rien n'a levé d'erreur, et le rapport par catégorie affiche les types : « Volume Discount » cinq fois, « Seasonal Discount » trois fois. Le correctif nomme les colonnes :

```sql
INSERT INTO labo03.promotion_historique
       (specialofferid, description, discountpct, type, category, startdate, enddate)
SELECT specialofferid, description, discountpct, type, category, startdate, enddate
FROM sales.specialoffer;
```

Les catégories redeviennent « Customer », « No Discount » et « Reseller ». Les lignes archivées de travers depuis la recréation se réparent par un `UPDATE` qui échange les deux colonnes, `SET type = category, category = type`, limité à ces lignes : PostgreSQL évalue toutes les expressions du `SET` sur la ligne d'avant, et l'échange se fait en une commande.

*Théorie : `Theorie_03_SQL.md`, section 4.*

## Questions à développer

A. Un entrepôt charge des millions de lignes d'un coup, depuis des sources qu'on peut contrôler avant le chargement. Vérifier l'unicité à chaque écriture demanderait un index ou une recherche par ligne, et une table ordinaire de Snowflake n'a pas d'index, d'après sa documentation (Cours 10) : le contrôle ralentirait tous les chargements pour protéger contre une erreur qu'un test attrape à moindre coût. La déclaration garde une valeur réelle : elle documente le modèle, les outils de modélisation et de visualisation la lisent pour relier les tables, et, marquée `RELY`, elle autorise l'optimiseur à retirer une jointure inutile, toujours d'après sa documentation. Ce choix déplace la responsabilité vers l'équipe qui charge, d'autant plus qu'une contrainte `RELY` démentie par les données expose à des résultats inattendus, que la même documentation met à sa charge. Elle doit rendre ses chargements rejouables, contrôler l'unicité de la source



© 2026 iAlexMG. Tous droits réservés.

avant d'écrire, tester la table après chaque chargement (clé unique, colonnes obligatoires présentes, clés étrangères qui trouvent leur ligne) et bloquer la publication quand un test échoue. Ce contrôle vit dans la chaîne de transformation, là où la formation Analytics Engineering le place avec ses tests, et jamais dans les rapports, qui arrivent trop tard. DuckDB, qui applique les clés, montre que ce choix n'est pas imposé par la nature d'un entrepôt. Même là, le test reste utile : l'entrepôt de la formation n'a déclaré aucune clé, et une clé qu'on n'a pas déclarée ne protège rien.

*Théorie : `Theorie_03_SQL.md`, section 3.*

B. Sur le volume, `DELETE` travaille ligne par ligne et reste lent sur des millions de lignes, alors que `TRUNCATE` vide la table d'un bloc. Sur les clés étrangères, `DELETE` refuse seulement les lignes encore référencées, `TRUNCATE` refuse la commande entière, et son `CASCADE` viderait aussi les tables qui la référencent. Sur les transactions, les deux s'annulent sur PostgreSQL et DuckDB, mais `TRUNCATE` appartient au DDL, et sur un moteur où le DDL est validé d'office, il ne faut pas compter sur le `ROLLBACK`. Sur ce qu'on garde, `DELETE` garde tout ce que son `WHERE` n'a pas visé ; `TRUNCATE` ne garde rien, sauf la structure. `TRUNCATE` est le bon choix pour vider une table de préparation qu'on recharge en entier chaque nuit. Ce serait une faute pour « recharger un mois » dans une table de faits de cinq ans, ou avec `CASCADE` sur une dimension, qui emporterait les faits qui la référencent.

*Théorie : `Theorie_03_SQL.md`, sections 5 et 8.*

C. Sur PostgreSQL ou DuckDB, si les trois étapes tournent dans une même transaction, l'échec du `NOT NULL` annule tout : après le `ROLLBACK`, la colonne n'existe pas, et la base est exactement dans son état de départ. Ce que le moteur oblige à prévoir, c'est d'écrire la migration entre un `BEGIN` et un `COMMIT`, puis de corriger les données et de relancer ; lancée en autovalidation, la migration laisserait le même état intermédiaire que sur Snowflake. Sur Snowflake, chaque DDL valide d'office : l'ajout de la colonne est validé, l'`UPDATE` aussi, et l'échec de la troisième étape laisse une colonne ajoutée, remplie en partie et sans `NOT NULL`. Il faut donc prévoir des étapes rejouables (`ADD COLUMN IF NOT EXISTS`), un contrôle avant la dernière étape, qui compte les absences et arrête tout s'il en trouve, un script de retour arrière (`DROP COLUMN`) ou une copie de la table prise juste avant, et une fenêtre où personne ne lit la table à moitié migrée.

*Théorie : `Theorie_03_SQL.md`, section 8.*

## Mises en situation

M1. Je choisis la suppression et la réinsertion des sept jours, dans une transaction, précédées d'un contrôle d'unicité du fichier. Le fichier fait autorité sur sa fenêtre : ce qu'il contient est la vérité des sept derniers jours, y compris ce qu'il ne contient plus. Premier cas, la livraison retirée par la ferme : la suppression de la fenêtre l'efface, alors qu'un `MERGE` ordinaire la laisserait en place, faute de ligne source pour la viser. Deuxième cas, le fichier livré deux fois : la seconde passe supprime et réinsère les mêmes lignes, et l'état final est identique ; le `MERGE` serait lui aussi rejouable. Troisième cas, deux lignes pour la même ferme et le même jour : le `MERGE` échouerait sur PostgreSQL ou Snowflake, ce qui est sain, mais passerait sur DuckDB en gardant l'une des deux ; la réinsertion, elle, les écrirait toutes les deux, et un entrepôt qui n'applique pas la clé primaire, comme Snowflake, ne l'en empêcherait pas. D'où le contrôle d'unicité du fichier, qui bloque le chargement avant la transaction, quel que soit le moteur. Sur Snowflake, la table de préparation se crée avant le `BEGIN`, pour qu'aucun DDL ne valide la transaction à mi-chemin. Le `MERGE` redevient le meilleur choix quand la source est un flux de changements plutôt qu'une fenêtre complète.

*Théorie : `Theorie_03_SQL.md`, sections 5, 6 et 8.*

M2. L'unicité ne vit plus dans le moteur, elle doit vivre dans la chaîne de chargement, à trois endroits. Avant d'écrire : le chargement arrive dans une table de préparation, où une requête de contrôle vérifie qu'aucune clé client n'apparaît deux fois. Pendant l'écriture : un `MERGE` sur la clé, qui échoue de lui-même sur



© 2026 iAlexMG. Tous droits réservés.

une source en double et ne crée pas de doublon quand on le rejoue. Après l'écriture : le même contrôle sur la dimension, parce qu'un autre traitement peut y écrire. Un chargement qui échoue au contrôle n'est pas publié : la dimension garde sa version de la veille, les lignes fautives partent dans une table de quarantaine, et quelqu'un est alerté. Je refuse le dédoublonnage automatique et silencieux ; s'il faut une règle, « la fiche la plus récente gagne » par exemple, elle est écrite, appliquée dans la requête source et revue. Il reste à réparer les 2 % existants : désigner la fiche qui survit pour chaque doublon, rebâtir la dimension, et vérifier que les faits pointent vers les bonnes clés.

*Théorie : `Theorie_03_SQL.md`, sections 3 et 6.*

M3. À 10 h, la vue montre les absences saisies jusqu'à 10 h ; la table de travail montre l'état de la veille au soir. La vue coûte quatre minutes à chaque consultation, multipliées par le nombre de directions qui l'ouvrent en même temps le matin ; la table coûte quatre minutes une fois par nuit, et se lit ensuite instantanément. Le jour où la requête contient une erreur, la vue la propage à tout le monde à la seconde, mais une seule correction la répare partout ; la table garde l'erreur jusqu'au prochain recalcul, et sa correction exige de relancer le calcul. Je choisis la table de travail recalculée la nuit, avec une colonne qui affiche la date et l'heure du calcul, parce que l'usage est la lecture du matin sur les jours précédents et que quatre minutes d'attente par direction usent la confiance dans l'outil. Si les directions ont besoin des absences du jour même, j'ajoute une petite vue limitée à la journée en cours, qui reste rapide.

*Théorie : `Theorie_03_SQL.md`, sections 2 et 7.*

M4. `(avancé)` Je ne transforme pas la colonne en place : un changement de type est refusé tant que les vues la lisent, et, sur une grosse table, il la réécrit en bloquant les lectures pendant ce temps (Cours 11). Je procède en sept étapes. Un : inventorier les formes présentes, par des requêtes à expressions régulières (des chiffres seuls, des chiffres suivis de « kg », « inconnu », le reste), et les compter. Deux : ajouter une colonne `poids_kg NUMERIC(8,2)`, facultative. Trois : la remplir par un `UPDATE` qui retire le suffixe et convertit ce qui a la bonne forme, par `CASE`, `REPLACE` et `TRIM`, dans une transaction, en vérifiant que le nombre de lignes converties égale l'inventaire et en listant ce qui résiste avant de valider. Quatre : faire écrire les deux colonnes par l'application pendant la transition. Cinq : basculer les deux vues sur `poids_kg`, par un `DROP VIEW` et un `CREATE VIEW` dans une même transaction, parce que `CREATE OR REPLACE VIEW` refuse de changer le type d'une colonne de vue et que le DDL de PostgreSQL s'annule en bloc. Six : comparer le total facturé d'un mois passé avant et après la bascule. Sept : retirer l'ancienne colonne quelques semaines plus tard. Chaque étape a son retour arrière : la nouvelle colonne se supprime, les vues se recréent sur leur ancienne définition dans une transaction, et l'ancienne colonne reste intacte jusqu'à la toute fin.

*Théorie : `Theorie_03_SQL.md`, sections 2, 7 et 8.*

M5. Dans les cinq premières minutes, j'arrête l'hémorragie : le site passe en maintenance, ou la prise de commandes est coupée. Je vérifie si l'employé est encore dans une transaction ouverte, auquel cas un `ROLLBACK` règle tout ; en autovalidation, la commande est validée. Je note l'heure exacte, 14 h 05, et je n'écris plus rien dans la table sans plan. Dans l'heure, je reconstitue les prix : restauration de la sauvegarde à l'instant d'avant l'erreur, sur un serveur à part, puis un `UPDATE … FROM` des prix restaurés vers la production, par identifiant de produit, dans une transaction, avec un compte rendu qui doit égaler le nombre de produits. Les commandes passées à 9,99 $ relèvent d'une décision d'affaires, et sans doute juridique, que je ne prends pas seul. Si la même erreur frappait la table des prix de l'entrepôt Snowflake, l'urgence serait moindre, puisque ce sont des rapports et non des ventes, et la réparation plus rapide : le voyage dans le temps relit la table telle qu'elle était avant la commande fautive (Cours 11), sans restauration. Ensuite, trois changements : personne n'écrit à la main dans la base de production, les corrections passent par des scripts relus ; les outils de requête sur la production s'ouvrent sans autovalidation, ou avec un compte en lecture seule par défaut ; et la restauration à un instant donné est testée régulièrement, pas découverte le jour de la panne.

*Théorie : `Theorie_03_SQL.md`, sections 5 et 8.*



© 2026 iAlexMG. Tous droits réservés.